

Clase 4

Crear agentes de IA con APIs gratuitas

Guía de trabajo del aprendiz · Gemini y Cohere

Duración: 4 horas (8:00 am – 12:00 m)

Pablo Andrés Carvajal — Desarrollador Full Stack

pablocarvajal.dev

Nombre del aprendiz: _____

¿Qué sabré hacer al terminar esta guía?

- Explicar qué diferencia un **agente** de un simple chatbot.
- Construir un chatbot con **memoria de conversación** (Gemini).
- Darle **herramientas** a un agente (function calling / tool use).
- Construir un **agente RAG** que responde con documentos propios (Cohere).
- Manejar mis claves de forma segura y entender los límites y costos por token.

LA GRAN INTEGRACIÓN

Esta clase reúne todo el curso: los **tokens** y **embeddings** de la Clase 2, el **RAG**, y las **APIs** gratuitas de la Clase 0 se combinan para crear agentes que funcionan de verdad. Pasamos de "chatear con un modelo" a construir asistentes con memoria, herramientas y datos propios.

DEMOS INTERACTIVOS DE ESTA CLASE

Prueba modelos y prompts en vivo antes de construir tu agente:

- **Playground de Gemini (ajusta la temperatura):** [Google AI Studio](#)
- **Playground y embeddings de Cohere:** [Cohere Dashboard](#)
- **Cómo la temperatura cambia la respuesta:** [Transformer Explainer](#)

Todas juntas en el hub del curso: [recursos/demos-interactivos.html](#)

1. ¿Qué es un agente de IA?

Un **chatbot** simple recibe un mensaje y responde: nada más. Un **agente** va mucho más allá: **razona, decide y actúa** para cumplir un objetivo, apoyándose en recursos externos. Es la diferencia entre alguien que solo conversa y alguien que resuelve tareas.

Anatomía de un agente

Un agente se compone de cuatro piezas:

1. **LLM (el cerebro):** decide qué hacer y genera lenguaje. En este curso usamos Gemini y Cohere.
2. **Memoria:** recuerda la conversación y los hechos importantes (historial de chat + RAG).
3. **Herramientas (tools):** funciones que el agente puede llamar: calcular, buscar en internet, consultar una base de datos, enviar un correo, ejecutar código...
4. **Bucle de razonamiento:** el ciclo *pensar* → *actuar* → *observar el resultado* → *repetir* hasta resolver la tarea.

YA CONOCES UN AGENTE

OpenCode (Clase 0) es exactamente esto: un LLM + herramientas (leer/escribir archivos, ejecutar comandos) + memoria del proyecto. Hoy construimos nuestra propia versión, más sencilla.

El patrón ReAct (Reason + Act)

La mayoría de los agentes siguen un ciclo en el que **alternan pensamiento y acción** hasta tener todo lo necesario para responder:

```
Pensamiento: necesito saber el clima de hoy.  
Acción:      llamar herramienta buscar_clima("Piedecuesta")  
Observación: "24°C, soleado"  
Pensamiento: ya tengo la info, puedo responder.  
Respuesta:   "Hoy en Piedecuesta hay 24°C y está soleado."
```

2. Proyecto 1 — Chatbot con memoria (Gemini)

Un modelo por defecto **no recuerda** el mensaje anterior: cada petición es independiente. La "memoria" se logra **enviándole el historial completo** en cada turno. La librería de Gemini nos facilita esto con `start_chat()`, que guarda el historial automáticamente y se lo reenvía al modelo.

Primero instalamos las dependencias:

```
pip install google-generativeai python-dotenv
```

El programa completo (archivo `clase-4/codigo/1_chatbot_memoria.py`), comentado paso a paso:

```

import os
from dotenv import load_dotenv
import google.generativeai as genai

load_dotenv() # carga las claves del .env
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

modelo = genai.GenerativeModel(
    "gemini-1.5-flash",
    # El system_instruction define la personalidad y las reglas del agente.
    system_instruction=(
        "Eres un tutor amable del SENA. Respondes de forma clara y breve, "
        "con ejemplos sencillos y en español."
    ),
)

# start_chat mantiene automáticamente el historial => MEMORIA.
chat = modelo.start_chat(history=[])

print("Tutor IA listo (escribe 'salir' para terminar)\n")
while True:
    mensaje = input("Tú: ").strip()
    if mensaje.lower() == "salir":
        break
    respuesta = chat.send_message(mensaje) # envía el turno + el historial
    print("Tutor:", respuesta.text, "\n")

```

Cómo demostrar que hay memoria

Escribe estos dos mensajes seguidos y observa la segunda respuesta:

Tú escribes	El tutor responde
Me llamo Andrés.	Mucho gusto, Andrés.
¿Cómo me llamo?	Te llamas Andrés. ✅ (recuerda porque enviamos el historial)

EL TRUCO MÁS PODEROSO

El `system_instruction` define la **personalidad y las reglas** del agente. Es la herramienta más poderosa y barata para controlarlo: cambia esa frase y tendrás un agente completamente distinto (un chef, un abogado, un tutor de inglés...).

3. Proyecto 2 — Agente con herramientas (function calling)

Los LLMs son **malos en matemáticas exactas** y **no conocen datos en tiempo real**. La solución es darles **herramientas**: funciones de Python que ellos deciden cuándo llamar. Esto se conoce como **function calling** o **tool use**.

Lo clave: una herramienta puede ser **cualquier función de Python**. El **docstring** (la descripción entre comillas triples) es fundamental, porque el modelo lo **lee** para saber qué hace la función y cuándo conviene usarla.

El programa completo (archivo `clase-4/codigo/2_agente_herramientas.py`):

```
import os
from dotenv import load_dotenv
import google.generativeai as genai

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

# 1. Definimos las herramientas como funciones normales de Python.
def calcular(operacion: str) -> str:
    """Evalúa una operación matemática escrita como texto. Ej: '15 * 23 + 7'."""
    try:
        # eval() se usa aquí SOLO con fines educativos.
        return str(eval(operacion, {"__builtins__": {}}, {}))
    except Exception:
        return "No pude calcular eso."

def clima(ciudad: str) -> str:
    """Devuelve el clima actual (simulado) de una ciudad dada."""
    datos = {"piedecuesta": "24°C, soleado", "bogota": "14°C, lluvioso"}
    return datos.get(ciudad.lower(), "No tengo datos de esa ciudad.")

# 2. Le entregamos las herramientas al modelo.
modelo = genai.GenerativeModel("gemini-1.5-flash", tools=[calcular, clima])
chat = modelo.start_chat(enable_automatic_function_calling=True)

# 3. El modelo DECIDE solo cuándo usar cada herramienta.
print(chat.send_message("¿Cuánto es 1520 * 34 + 89?").text)
print(chat.send_message("¿Qué clima hace en Piedecuesta?").text)
```

Lo importante que verá el grupo cuando corra el programa:

- El aprendiz pregunta en **lenguaje natural**, sin decir qué función usar.
- El modelo **entiende** por sí solo que debe usar `calcular` o `clima`.
- Llama la función, recibe el resultado y **responde con él**.

ASÍ SE CONECTAN LOS LLMS AL MUNDO REAL

Una herramienta puede consultar una base de datos, mandar correos o llamar la API de tu empresa. Con function calling, el LLM deja de ser solo un "hablador" y empieza a **actuar**.

INDEPENDIENTE DEL PROVEEDOR

Cohere también soporta *tool use* con su API de chat (`co.chat` con el parámetro `tools`); se declaran las herramientas y el modelo pide ejecutarlas. La idea es idéntica, lo que muestra que **el concepto de agente no depende del proveedor**.

4. Proyecto 3 — Agente RAG con datos propios (Cohere)

Unimos el **RAG** (Retrieval-Augmented Generation) de la Clase 2 con un agente que **responde solo con nuestros documentos**. Un caso real muy útil: un asistente del reglamento del SENA que contesta dudas de los aprendices.

RAG funciona en tres pasos:

1. **Vectorizar** nuestros documentos una sola vez (embeddings).
2. **Buscar** (retrieval) los fragmentos más parecidos a la pregunta.
3. **Generar** (generation) la respuesta usando SOLO esos fragmentos.

Instalamos las dependencias:

```
pip install cohere numpy python-dotenv
```

El programa completo (archivo `clase-4/codigo/3_agente_rag.py`), comentado:

```

import os, cohere, numpy as np
from dotenv import load_dotenv

load_dotenv()
co = cohere.Client(os.getenv("COHERE_API_KEY"))

# 1. Base de conocimiento: chunks (fragmentos) de nuestros documentos.
documentos = [
    "El curso de IA del SENA Piedecuesta se dicta los sábados de 8am a 12m.",
    "Se requiere el 80% de asistencia para poder certificarse en el curso.",
    "El instructor del curso es Andrés Carvajal.",
    "Las APIs usadas en el curso son Cohere y Gemini, ambas con planes gratuitos.",
]

# 2. Vectorizamos la base UNA sola vez.
emb_docs = co.embed(texts=documentos, model="embed-multilingual-v3.0",
                    input_type="search_document").embeddings

def similitud(a, b):
    """Similitud coseno entre dos vectores (1 = idénticos)."""
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def responder(pregunta: str) -> str:
    # 3. RETRIEVAL: buscar los 2 chunks más relevantes.
    emb_p = co.embed(texts=[pregunta], model="embed-multilingual-v3.0",
                    input_type="search_query").embeddings[0]
    ranking = sorted(range(len(documentos)),
                    key=lambda i: similitud(emb_p, emb_docs[i]), reverse=True)
    contexto = "\n".join(f"- {documentos[i]}" for i in ranking[:2])

    # 4. GENERATION: responder usando SOLO el contexto recuperado.
    prompt = (f"Responde usando ÚNICAMENTE esta información.\n"
             f"Si la respuesta no está ahí, di que no tienes ese dato.\n\n"
             f"Información:\n{contexto}\n\nPregunta: {pregunta}")
    return co.chat(message=prompt, model="command-r").text

print(responder("¿A qué hora es el curso?"))
print(responder("¿Quién es el instructor?"))
print(responder("¿Cuánto cuesta un carro?")) # → debe decir que no lo sabe

```

Puntos para discutir con el grupo:


```
pip install streamlit
# app.py con la lógica del agente + st.chat_input()
streamlit run app.py
```

6. Proyecto final y rúbrica

Cada aprendiz (o pareja) construye **su propio agente**, eligiendo una de estas opciones:

- Un **chatbot con personalidad** (definida en el `system_instruction`) para un tema, o
- Un **agente con una herramienta** propia (function calling), o
- Un **agente RAG** con sus propios documentos.

Rúbrica de evaluación:

Criterio	Puntos
El agente funciona y responde	40
Usa memoria, herramienta o RAG	30
Maneja las claves de forma segura (.env + .gitignore)	15
Presentación y explicación clara de cómo funciona	15
Total	100

Actividades de refuerzo

Realiza estas actividades para afianzar lo aprendido. Puedes escribir tus respuestas en los espacios en blanco.

1 Chatbot vs. agente Fácil

Explica con tus propias palabras dos diferencias entre un chatbot simple y un agente. Nombra las cuatro piezas de la anatomía de un agente.

2 Dale personalidad a tu tutor Fácil

Ejecuta `1_chatbot_memoria.py` y cambia el `system_instruction` por otra personalidad (por ejemplo, un chef o un guía turístico). Anota qué frase usaste y cómo cambió el comportamiento.

3 Comprueba la memoria Media

Con el chatbot en marcha, dile tu nombre y una comida favorita en mensajes separados. Luego pregúntale por ambos datos. Anota si los recordó y explica por qué funciona la memoria.

4 Tu propia herramienta Media

En `2_agente_herramientas.py` agrega una nueva función-herramienta (por ejemplo, `convertir_moneda(pesos)` o `hora_actual()`) con su docstring, inclúyela en la lista `tools= [...]` y haz que el modelo la use. Anota tu función.

5 RAG con tus datos Reto

En `3_agente_rag.py` reemplaza la lista `documentos` por 5 datos reales sobre ti, tu barrio o tu emprendimiento. Hazle 3 preguntas al agente (una cuya respuesta NO esté en los datos) y anota qué respondió.

6 Proyecto final: tu propio agente Reto

Construye **tu propio agente** eligiendo una opción: (a) un chatbot con personalidad, (b) un agente con una herramienta propia, o (c) un agente RAG con tus documentos. Recuerda guardar tus claves en `.env` y crear el `.gitignore`. Prepárate para explicar cómo funciona (rúbrica de la sección 6). Describe aquí tu idea:

Lista de verificación

☐ Sé qué distingue a un agente de un chatbot.

- ☐ Puedo darle memoria a un modelo con `start_chat()` .
- ☐ Puedo darle herramientas (function calling) a un agente.
- ☐ Puedo construir un agente RAG que responde con mis datos.
- ☐ Guardo mis claves en `.env` y tengo un `.gitignore` .
- ☐ Entiendo que las APIs gratuitas tienen límites y que se cobra por token.
- ☐ Elegí y describí mi proyecto final.

PARA SEGUIR APRENDIENDO

Frameworks de agentes: LangChain, LlamaIndex, CrewAI. · **Bases de datos vectoriales:** Chroma, FAISS, Pinecone. · **Modelos locales gratis:** Ollama (corre LLMs en tu propio PC). · **Documentación oficial:** Google AI Studio y Cohere Docs.